Exercise Set 3: Copy Control (Rule of Three)

Basic Copy Control Implementation

Exercise 1 Easy

Create a class `String` that manages a dynamically allocated character array. Implement the copy constructor, copy assignment operator, and destructor to perform deep copying.

> **Expected Implementation Details:**
>
> The copy constructor should allocate new memory and copy the contents character by character. The assignment operator should handle self-assignment, delete old memory, allocate new memory, and copy contents. The destructor should deallocate the dynamic memory. Test with code like: `String s1("Hello"); String s2(s1); String s3 = s2;` Each object should have its own copy of the string data.

Exercise 2 Easy

Design a class `IntArray` that wraps a dynamic integer array. Implement proper copy control to ensure deep copying of the array contents.

> **Expected Implementation Details:**
>
> Include a size member to track array length. The copy constructor should allocate a new array of the same size and copy all elements. The assignment operator must prevent memory leaks and handle arrays of different sizes. Provide methods like `set(index, value)` and `get(index)`. When one object is modified, other copies should remain unchanged.

Exercise 3 Easy

Implement a class `Person` with a dynamically allocated name (C-string) and age. Write copy control members to properly manage the dynamic memory.

> **Expected Implementation Details:**
>
> The name should be stored as a `char*`. Copy constructor and assignment operator should perform deep copy of the name string. Include proper null checks and handle empty names. Test by creating copies and changing names - each object should maintain its own name independently.

Exercise 4 Easy

Create a class `SharedResource` that uses shallow copying with reference counting. Implement copy constructor, assignment operator, and destructor to manage shared ownership.

> **Expected Implementation Details:**
>
> Use a pointer to the resource and a pointer to a reference count (both shared among copies). Copy constructor increments the count, destructor decrements it and deletes when count reaches zero. Assignment operator should handle both the old and new resources correctly. Multiple objects should share the same resource until the last one is destroyed.

Exercise 5 Easy

Design a class `FileHandler` that manages a file pointer. Implement copy control to ensure only one object owns the file at a time (transfer of ownership).

> **Expected Implementation Details:**
>
> Store a `FILE*` pointer. Copy constructor should transfer ownership from source to destination (set source pointer to nullptr). Assignment operator should close any existing file before taking ownership. Destructor closes the file if owned. This implements a simple unique ownership model without C++11 features.

Container and Data Structure Implementations

Exercise 6 Intermediate

Implement a class `LinkedList` with proper copy control. The list should support basic operations and ensure deep copying of all nodes.

> **Expected Implementation Details:**
>
> Define a `Node` structure with data and next pointer. Copy constructor should create new nodes for each element in the source list, maintaining the same order. Assignment operator must delete all existing nodes before copying. Include methods: `push_front()`, `pop_front()`, `size()`, and `display()`. Test by creating a list, making copies, and modifying each independently - changes to one list should not affect others.

Exercise 7 Intermediate

Create a class `Stack` using a dynamic array implementation. Implement copy control to handle deep copying and dynamic resizing.

> **Expected Implementation Details:**
>
> Store current size, capacity, and a pointer to dynamic array. Copy constructor should copy only the used portion of the array. Assignment operator should optimize by reusing existing memory if capacity is sufficient. Include `push()`, `pop()`, `top()`, and automatic resizing when full. Each stack copy should be independent with its own memory.

Exercise 8 Intermediate

Design a class `Matrix` with dynamic 2D array allocation. Implement complete copy control for deep copying of the matrix data.

> **Expected Implementation Details:**
>
> Use a pointer-to-pointer (`int**`) for 2D dynamic allocation. Store rows and columns count. Copy constructor must allocate new 2D array and copy all elements. Assignment operator should handle matrices of different dimensions. Include methods for element access, addition, and display. Ensure proper cleanup of all allocated rows in destructor.

Exercise 9 Intermediate

Implement a class `BinaryTree` with proper copy control. Each node contains an integer and pointers to left and right children.

> **Expected Implementation Details:**
>
> Define a `TreeNode` with data, left, and right pointers. Copy constructor should perform a deep copy of the entire tree structure (consider using a recursive helper function). Assignment operator must delete the old tree before copying. Include methods: `insert()`, `search()`, `inorder()`. Test by creating a tree, copying it, and modifying each independently.

Exercise 10 Difficult

Create a class `Graph` using adjacency list representation. Implement copy control to deep copy the entire graph structure.

> **Expected Implementation Details:**
>
> Use an array of linked lists or vector of lists to represent edges. Each vertex has a list of adjacent vertices. Copy constructor must recreate the entire adjacency structure. Include methods: `addVertex()`, `addEdge()`, `removeEdge()`, `display()`. Handle both directed and undirected graphs. Ensure that modifications to a copied graph don't affect the original.

Advanced Copy Control Scenarios

Exercise 11 Easy

Design a class `Database` that contains an array of `Record` pointers. Implement copy control to properly manage the array and all pointed-to records.

> **Expected Implementation Details:**
>
> Each `Record` should have dynamically allocated fields. The `Database` copy constructor must create new `Record` objects (not just copy pointers). Assignment operator should delete all existing records before copying. Include methods to add, remove, and search records. Test that modifying records in one database doesn't affect copies.

Exercise 12 Intermediate

Implement a class `Polynomial` that stores coefficients in a dynamic array. Support copy control and ensure proper handling of polynomials of different degrees.

> **Expected Implementation Details:**
>
> Store degree and coefficient array. Copy constructor should allocate array based on source polynomial's degree. Assignment operator must handle polynomials of different sizes efficiently. Include methods: `setCoefficient()`, `evaluate(x)`, `add()`. The destructor should free the coefficient array. Each polynomial copy should be completely independent.

Exercise 13 Intermediate

Create a class `TextEditor` that maintains a dynamic array of lines (each line is a dynamic string). Implement complete copy control.

> **Expected Implementation Details:**
>
> Use `char**` to store lines, with each line being a dynamically allocated string. Copy constructor must deep copy all lines. Assignment operator should properly clean up old lines and copy new ones. Include methods: `addLine()`, `deleteLine()`, `editLine()`, `display()`. Handle empty lines and varying line lengths correctly.

Exercise 14 Difficult

Design a class `SparseMatrix` that stores only non-zero elements using a linked structure. Implement copy control for this complex data structure.

> **Expected Implementation Details:**
>
> Use a linked list of nodes where each node stores row, column, and value. Copy constructor must create new nodes for all non-zero elements. Assignment operator should efficiently handle matrices of different dimensions and sparsity. Include methods: `set(row, col, value)`, `get(row, col)`, `multiply()`. Ensure proper memory management for the linked structure.

Exercise 15 Difficult

Implement a class `MemoryManager` that provides custom allocation for other objects. Include copy control that properly handles the managed memory pool.

> **Expected Implementation Details:**
>
> Maintain a large memory buffer and a free list. Copy constructor should duplicate the entire memory state, including allocated and free blocks. Assignment operator must handle different pool sizes. Track allocations using a linked list of block headers. The destructor should ensure all managed memory is properly released. This is a complex exercise requiring careful pointer management.

Special Cases and Mixed Scenarios

Exercise 16 Easy

Create a class `CircularBuffer` with fixed size. Implement copy control ensuring the circular nature is preserved in copies.

> **Expected Implementation Details:**
>
> Store head and tail indices along with the buffer. Copy constructor should copy the logical contents (not just the array) maintaining the circular order. Include methods: `enqueue()`, `dequeue()`, `isFull()`, `isEmpty()`. When copying a partially filled buffer, ensure the copy has the same logical content in the correct order.

Exercise 17 Intermediate

Design a class `HashTable` using separate chaining. Implement copy control to deep copy all chains and elements.

> **Expected Implementation Details:**
>
> Use an array of linked list pointers for chains. Copy constructor must recreate all chains with new nodes. Assignment operator should properly delete old chains before copying. Include methods: `insert(key, value)`, `search(key)`, `remove(key)`. Handle hash collisions and ensure that operations on a copy don't affect the original table.

Exercise 18 Intermediate

Implement a class `Trie` (prefix tree) for string storage. Provide complete copy control for the tree structure.

> **Expected Implementation Details:**
>
> Each node contains an array of child pointers (26 for lowercase letters) and an end-of-word flag. Copy constructor must recursively copy the entire trie structure. Include methods: `insert(word)`, `search(word)`, `startsWith(prefix)`. The destructor must recursively delete all nodes. Ensure copied tries are completely independent.

Exercise 19 Difficult

Create a class `ThreadPool` simulator that manages a collection of `Task` objects. Implement copy control considering the complex ownership relationships.

> **Expected Implementation Details:**
>
> Store a queue of `Task` pointers and an array of `Worker` objects. Copy constructor should deep copy all tasks but create new workers. Assignment operator must handle active vs. pending tasks differently. Include proper synchronization placeholders (even though actual threading isn't required). This tests understanding of complex object relationships and ownership.

Exercise 20 Difficult

Design a class `ExpressionTree` that represents mathematical expressions. Implement copy control to properly duplicate the tree structure and support expression evaluation.

> **Expected Implementation Details:**
>
> Nodes can be operators (+, -, *, /) or operands (numbers). Each operator node has left and right children. Copy constructor must recursively copy the entire expression tree. Include methods: `evaluate()`, `printInfix()`, `printPostfix()`. The assignment operator should handle trees of different complexities. Test with expressions like "3 + 4 * 2" represented as trees.